

Babilonia: probabilistic coinjoin and covert betting

waxwing

July 12, 2026

Version 1.0

Contents

1	Introduction	2
1.1	The pot-sweetener incentive	3
2	Earlier work	3
2.1	Collaborative transactions without balance preservation - interesting?	4
3	The verifiable onchain coin-flip	5
3.1	Comparison with GTT (2026) [9]	5
3.2	An outline of the protocol	6
3.3	The protocol as a sequence of messages	9
3.4	Extension to other payout probabilities	10
3.5	Payment-shaped transactions	10
3.6	A pot-sweetener example	12
4	Construction of the proof π_A	13
4.1	The broken case: $f(t) = t$	13
4.2	Squaring pad: $f(t) = t^2$	13
4.2.1	Squaring proof construction	14
4.2.2	Squaring proof verification	14
4.2.3	Squaring proof security	15
4.3	ZK-friendly hash pad: $f = \text{Poseidon}$	15
5	Cooperative Alice-payout completion	16
6	Network communication layer	16
6.1	BIP324 decoy packets	16
6.2	Discovery and Sybil attacks	17
7	Example implementation	17
8	Other similar ideas	17
9	Appendix A: Reduction of the squaring proof to square-DDH	20

9.1	Remarks	21
9.2	Tightness reduction	21
10	Appendix B: Mathematics of betting on coinflips	23
10.1	Fixed-fraction betting (Kelly)	23
10.2	Fixed-amount betting (additive walk)	23
10.3	Some vague thoughts on the effective cost of this as a coinjoin strategy	24

1. Introduction

There are essentially two approaches to online privacy. The first is *uniformity*, probably best exemplified by mixnets [1] but also seen in most other serious attempts to create privacy: they usually involve coordination by a central entity and require the actors in the system to behave identically or near-identically. A good example is the equal-output coinjoin design exemplified by Samourai and Wasabi [3, 4]. These approaches are attractive in enabling quantifiable anonymity sets.

There are numerous issues with this approach, from disadvantages with central coordination to fragility with respect to other participants non-uniform behavior and the one that is unlikely to be captured in academic analysis:

Recognizably privacy seeking behavior is societally disadvantaged.

That last point more than anything leads to a search for techniques in which privacy is a *secondary* goal. This is not to make the fallacious claim that “anonymity sets are larger when you don’t choose to prioritize privacy”; this is trivially false, because standard activities (such as payments) don’t have indistinguishable behaviour between parties. The idea is subtler, and is our “second approach”: instead of trying to create an unambiguous, quantifiable anonymity set, the user simply creates (intentionally or not) alternative plausible intentions for their actions. Note how this defends against a different attacker: to defend against an adversary who targets your ownership, you should obscure ownership. To defend against an attacker who targets behavior, you should obscure behavior. Both attackers could be a threat to your personal well-being. This ties back to “societally disadvantaged”.

For example, using Lightning [5] to buy coffee may have the intention of saving transaction fees or speeding up ‘confirmation’ but may create privacy as a byproduct. This line of reasoning has been, and should continue to be, researched and investigated (and indeed the Lightning case nicely illustrates that you generally will defend against the former attacker *as well as* the latter, that is, you will often obscure ownership too, at least partly). This reduces the societal disadvantage of general Bitcoin usage. If one focuses on on-chain Bitcoin the situation is clearly less bright, but this paper investigates an angle that is somewhat similar to Payjoin [6].

If we take coinjoins [2] as by definition onchain Bitcoin transactions in which there is more than one actor providing input funds, then Payjoins are a coinjoin variant in which a payment occurs, but here we propose a twist on that idea: in case the user does not actually want to spend money at a merchant’s shop, but is either willing or desiring to *gamble*¹, a similar outcome can be created not only with a trusted merchant, but also with a completely unknown counterparty,

¹Such a user should note the work of Bachelier and Einstein on random walks... and read Appendix B

assuming a completely trustless fair bet is possible. We will present here a protocol to do exactly that.

The same technique can be applied to private coinswaps [26]. Further development of that idea is deferred to other work.

If we want to create privacy against the second aforementioned type of attacker, that is one who targets behavior, then one could easily and correctly argue that *any* gambling activity has the desired effect: chips cashed out via the casino are apparently “winnings”, not outputs of mixing. This applies, e.g. to online bitcoin exchanges, etc. One could add any of a vast number of similar techniques, both online and offline. What is distinct here is that a *trustless* form of gambling does not require a central point of failure, with all the concomitant risks: trust in the entity, hacking risk, network monitoring, transaction graph being not distributed, performance problems etc. etc.

Finally let’s mention the steganographic aspect. First, as this author noted in a blog post from 2019 [28], the specific most useful effect of payjoin is that through the steganographic nature of such transactions — a deep confusion of CIOH-violating transactions with others makes blockchain analysis far less effective. Additionally, through a specific suggested mechanism here², though others exist, we can arrange for bet transaction negotiation to be completely unknown to anyone except the counterparties, and the onchain footprint to be indistinguishable from ordinary payments *on the happy path*. We don’t claim perfection in this regard, but interestingly we don’t have to, *if* a betting market of such bets already exists, the covertness of “looks like a payment” is only one of two layers of defense against a privacy adversary.

1.1. The pot-sweetener incentive

This feature is only briefly mentioned in the below protocol design (see Section 3.6), but it may be very important in creating participation. One bettor could incentivize any other bettor to take part by constructing the transactions so that the payouts are skewed slightly in their favor. This could occur in cases where Alice is looking mainly for a privacy effect, and Bob is more interested in the actual bet, i.e. gambling. This does *not* suffer from the same watermarking flaw as Joinmarket [29] in the specific sense that it is analogous to a merchant offering a discount for paying via coinjoin (that is, it changes the effective payment amount without even revealing the exact nature of the payment, let alone the betting function).

2. Earlier work

The first successful onchain gambling system was probably SatoshiDice [7]. In this system there was not a claim of trustlessness, but instead of *verifiable after-the-fact fairness*. Gamblers placed bets by paying to specific addresses owned by SatoshiDice, and could calculate after the fact (using blockchain-derived entropy; details and caveats omitted for brevity) whether they *should* have won or lost, and match that calculation against the outcome that was provided to them by the central entity.

Because of the control of funds by the central entity, a covert “looks like a typical payment” property was not achieved (nor, of course claimed); at least, if the transaction graph as a whole

²BIP324 [22] decoy packets; see Section 6.

is analyzed. Here we are referencing what was already pointed out in the Introduction: the CPOF has a multifaceted disadvantage from a privacy perspective.

Recent research by Kurbatov and later GTT ([8, 9]) investigated whether a fair coin-flip could be emulated on Bitcoin without complex onchain contracting and fully peer to peer, because no trust is required by the two parties in advance.

With regards to the former, note that Kurbatov’s starting idea is an analogy with a “thimbles game” (a.k.a. “shell game”) in which the player must guess under which cup (or “thimble”) the ball is placed. He used hashing to hide the dealer’s choice and made use of the ability to derive pubkeys that are hidden in hashed addresses (e.g. segwit pay-to-pubkey-hash) at the time of spending, so that “reveal pubkey” is atomic with “broadcast spend”. To convert this to a taproot setting, as we will do, we make use of adaptor signatures, though several other tweaks are needed.

When speaking of the later paper [9], it seems that it was motivated by the desire for the existence of probabilistic payments and swaps executed not only in onchain form but also over Lightning or perhaps similar second layers. We will discuss a little more about this paper in the next section, because it is a substantially more powerful construction than the one shown here. *This paper examines a concrete instantiation of the same general idea for onchain bets, but with a different extra property: maintaining a certain kind of privacy profile.*

Explaining further: the lack of any onchain contracts in what we describe here, replaced with offchain interactions using zero knowledge proofs, allows instantiation of a protocol with a steganographic footprint, so that the transactions can be interpreted by the outside world as normal peer to peer payments, or bets, without any emphasis on the privacy effect created, that is, the obfuscation of ownership of the resulting funds.

2.1. Collaborative transactions without balance preservation - interesting?

To state the obvious, this represents a radical change in the design, when compared with earlier coinjoin ideas, because here there is not an assumption that parties do not randomly gain or lose money, but rather only an assumption that the transfer of funds is fair in the statistical sense, and verifiably so, even against an adversarial and unknown counterparty.

Let’s consider the outcome of repeated games. Those not of a gambling bent or not familiar with random walks are likely unaware of the existing theory about this.

To avoid spamming up the paper with gambling theory, I’ll write a short “takeaways” here and defer (a little) more detail to Appendix B.

If the bet is a large fraction of your balance, consistently, over many bets, that’s a big problem: something called “asymmetric leverage” will *almost inevitably* eat up your funds over time (not that you can’t get lucky with a win streak of course, but there is a strong tendency to lose, which seems counterintuitive if each single bet is fair, right ...).

However this is easily avoided by simply betting small amounts, and by not altering bet size according to your current balance. Think, the “payments” you’re making are only say x sats, whether your balance is $20x$ or $200x$. This takes away the above effect; now your balance experiences a random walk, but doesn’t drift downwards (except from paying network tx fees).

You can imagine the volatility of your balance as comparable to what you might pay a coinjoin coordinator, except (a) it's statistical: you might even *gain* rather than lose and (b) what you're paying for is very different: a string of various payjoin-like transactions that don't look like coinjoins and cannot be flagged as such.

The biggest problem with this framing is: if you want these transactions to literally have the same distribution as other payments that exist, they cannot realistically be pinned to only very dusty payments. This tension is real; probably the realistic scenario is “mostly small, or very small, but randomly varying bet sizes”.

Is it a compelling idea for users? That seems debatable. The idea of “pot sweeteners” mentioned above may help. There are pros and cons.

But we emphasize again that, like payjoin, **these transactions break subset-sum analysis, and they look exactly like payments, because they are payments**.

3. The verifiable onchain coin-flip

Here we will describe how to make the trustless fair bet using cryptography. We follow on here from the reasoning of Kurbatov [8] and GTT [9], without retracing all the steps and arguments from those works in the interest of brevity.

However, it is briefly worth mentioning more about the latter.

3.1. Comparison with GTT (2026) [9]

This comparison is important, because in the most normal reading, [9] is a significantly more powerful construction than the one we describe here. The rough comparison is: that paper describes how to do fair lottery/random selection with arbitrarily small success probabilities over *swap* transactions, not joins (which also naturally integrates into Lightning), whereas this construction is described for joins and, as presented, only with 2 outcomes; we'll briefly describe how it can be extended to N outcomes, but N is realistically restricted to very small amounts, as our communication is linear in N . So why is the construction described here interesting? To quote from [9] :

On-chain cost and fees. A complete winning probabilistic swap execution involves exactly four on-chain transactions: two funding transactions (one per party), one payment-claim transaction by the dealer, and one reward-claim transaction by the party. Each transaction uses a standard locking script $\text{OR}(\text{pktmp}, T, \text{pk})$ as defined in Figure 2, which requires only a single signature for the claim path and a timelock check for the refund path. These scripts are already supported natively by Bitcoin and Litecoin Taproot via `OP_CHECKSIG` and `OP_CHECKLOCKTIMEVERIFY` (BIP 65), and do not require any additional opcodes or custom spending conditions beyond what standard atomic swaps already use [61]. Consequently, probabilistic swap transactions have the same on-chain structure as ordinary script-path Taproot spends.

The point is as much the structure, as the cost: 4 onchain transactions, **with exposed scripts**, does not create an ideal 2-party betting system if we want to have onchain patterns that look

like payments. We’ll see both in this section and later ones, that we can achieve a “fair bet” scenario with very lightweight ZKPs and only standard keypath spend taproot outputs, *and* transaction structures indistinguishable from ordinary payments (in the happy path), **and only 2 onchain transactions, not 4**³. If *these* aspects of the design are not important, it is likely that [9] is more interesting. Note that [9] does use a specific additional cryptographic feature to achieve its goals, namely a certain form of OPRF [21]. Note also that with more interaction, it should be possible for that construction to have “overlay” transactions to remove the onchain script footprint, too. The ZKPs are intrinsically more weighty than here, due to the nature of what is being proved (and to achieve the goal of arbitrarily small probabilities without linear communication blowup).

One more preamble: this section describes the mechanism and, at its end, the on-chain payment-shaping that gives the bets their covert footprint (Section 3.5); the network-layer covert-communication and discovery options are deferred to Section 6.

3.2. An outline of the protocol

Throughout, $\mathbb{G} = \langle G \rangle$ denotes the secp256k1 group of prime order p , and $\mathbb{F}_p$ its scalar field; $H(\cdot)$ is the BIP340 challenge hash.

If funds are deposited by both parties (henceforth: Alice as “dealer”, initiating the bet, and Bob as “player”, accepting the bet, but noting that there is no trust in the dealer) into a joint MuSig2 [10, 11] output, we must construct two outgoing transactions: a RefundTx and a PayoutTx, as alternative spending paths from what we will call U_1 , the utxo owned by the MuSig. Write P_A, P_B for Alice’s and Bob’s *funding* keys, so that U_1 ’s scriptPubkey is a keypath-only taproot output for the aggregate $P_{\text{agg}} = \text{KeyAgg}(P_A, P_B)$.

Next, to start the actual protocol, Alice randomly generates two secret values a_1, a_2 from the set of valid scalars in $\mathbb{F}_p$ where p is the order of the secp256k1 group. She sends $A_i = a_i G$, $i \in \{1, 2\}$ to Bob along with a PoK of each of these secret discrete logs using the vanilla method (Schnorr signature). We call A_i the “thimbles”.

The RefundTx is straightforward: an `nLockTime` field, value T_R , is set in this transaction which prevents its spending before a certain time in the future; note that many wallets set `nLockTime` to a very recent block for anti-fee sniping [24]. T_R will be set quite far out to ensure this backout is not possible until the contract completes.

This transaction RefundTx will reassign funds to their original owners and must be co-signed by Alice and Bob *before the funding transaction is signed*.

Once the funding transaction is signed by both parties, the next stage is the provision, by Bob, of a receiving pubkey used in constructing the taproot scriptPubKey in the PayoutTransaction. This will take the form:

$$K = W_b + A_y$$

The value $W_b = w_b G$ is a *fresh hidden claim key* chosen by Bob, secret to this transaction and

³it is more practical to call it 2.5 transactions, but theoretically it’s 2

never revealed to Alice. We fix the index convention used throughout: $c \in \{1, 2\}$ is *Alice's* secret choice, and $y \in \{1, 2\}$ is *Bob's* secret guess; Bob wins iff $y = c$. So A_y is the thimble corresponding to Bob's guess. His inability to know Alice's choice c is clearly not dependent on any assumption cryptographically, only on Alice's general security.

Bob will have to attach a proof of honest construction of this value K , which is a simple Σ -protocol. This will be called π_B .

Definition 1 (π_B : honest construction of the claim key). *Given public thimbles $A_1, A_2 \in \mathbb{G}$ and Bob's claim key $K \in \mathbb{G}$, π_B is a zero-knowledge proof for the relation*

$$\mathcal{R}_{\pi_B} = \left\{ ((K, A_1, A_2); (w_b, y)) : y \in \{1, 2\}, K = w_b G + A_y \right\}.$$

Equivalently, Bob knows the discrete log of $K - A_y$ with respect to G for one branch y (his guess), without revealing which. It is realised as a 1-of-2 CDS disjunction [13] of Schnorr proofs of knowledge of $\text{dlog}_G(K - A_1)$ or $\text{dlog}_G(K - A_2)$: soundness forces K to be A_1 or A_2 offset by a value Bob knows, and witness-indistinguishability hides y .

Once Alice has verified π_B she is ready to prepare the PayoutTx template; it has one input U_1 , and at least one output, paying to a scriptPubkey, which we will call O_K , whose taproot internal key is K , and which has one leaf: a relative-timelock reclaim path for Alice, of the form

$$\langle t_1 \rangle \text{ OP_CHECKSEQUENCEVERIFY OP_DROP } \langle P_A \rangle \text{ OP_CHECKSIG},$$

where t_1 is a relative timelock in blocks (BIP68). t_1 is small relative to T_R . Thus the honest winner spends O_K by the *key path* under K (an ordinary taproot payment, no script revealed), while Alice can reclaim after t_1 blocks via this single script leaf only if Bob never claims.

In a later section (5) we show how we can preserve the “payment-like” characteristic of this transaction even when the logic of the betting game gives the payout to Alice, that is, Bob has guessed incorrectly ($y \neq c$). An important note upfront: taproot's design [20] enforces that keypath-spends for cases where a script-path leaf exists are *not* distinguishable, even after the payment occurs, from keypath-spends from scriptPubKeys where no script-path spend existed, under the assumption that the observer does not know the original internal key used in constructing the output. This is because even pure keypath scriptPubkeys use a dummy script path formed by $H_{TapTweak}(P)$, and P cannot be extracted by the observer.

Other outputs on the transaction are needed (usually, exactly one) to make this a typical payment transaction pattern; the choice of this does not affect any aspect of the game's security. The mechanism for this is described later in this document (3.5).

With the full template of PayoutTx created, the next step is for Alice and Bob to start a MuSig2 negotiation, using the prepared nonces $R_{i,j}$ (which can be sent well in advance) as according to the standard MuSig2 workflow. After the nonce preparation, Alice can next create an *adaptor signature*, denoted σ' , and provide along with it an adaptor point T .

The concept is that there will be a ZKP π_A that will prove that T 's discrete log t is a kind of “decryption secret” that will allow Bob to calculate *one of* a_1, a_2 and thus get the full private key of K , but he will not be able to deduce which, until the adaptor signature (or “encrypted signature” a la Fournier [12]) is completed with the full signature, i.e. when the transaction is broadcast.

Notice a crucial game-theoretical aspect of this: the information hiding **has to work both ways**. This point was expanded on in some detail in GTT [9]. If Alice could deduce anything about Bob’s choice, encoded in K , she could choose to back out preferentially at this step of the protocol. And if Bob knew Alice’s choice, he could do the same at the next step (when he co-signs). Because Alice *doesn’t* know Bob’s choice at this step, while she *could* still get cold feet and fall back to Refund, she has no more incentive to back out now, than she did at the start.

When Bob receives this adaptor, a ciphertext ctxt , and the corresponding zero knowledge proof π_A (explained in a moment), he can verify the adaptor pre-signature:

$$\sigma'G \stackrel{?}{=} R_A - T + e\mu_AP_A, \quad e = H(R_{\text{agg}}, P_{\text{agg}}, m_{\text{payout}}),$$

...where R_A is Alice’s contribution to the aggregate nonce R_{agg} , and μ_A is Alice’s MuSig2 key-aggregation coefficient (we write μ_A , not a_A , to avoid clashing with the thimble scalars a_1, a_2); thus μ_AP_A is Alice’s delinearized key component and m_{payout} is the PayoutTx sighash. The check holds because Alice’s completed partial would satisfy $\sigma_AG = R_A + e\mu_AP_A$, and the adaptor partial is short by exactly t : $\sigma' = \sigma_A - t$, i.e. $\sigma'G = R_A - T + e\mu_AP_A$. See MuSig2 [10, 11] for the aggregation details, and [12] for adaptor (one-time verifiably-encrypted) signatures.

Additionally, he of course must verify π_A :

Definition 2 (π_A : well-formedness of the encrypted outcome). *Fix a pad function $f : \mathbb{F}_p \rightarrow \mathbb{F}_p$ (the “decryption” map; concrete choices in Section 4). Given public $(\text{ctxt}, T, A_1, A_2)$ with $\text{ctxt} \in \mathbb{F}_p$ and $T, A_1, A_2 \in \mathbb{G}$, π_A is a zero-knowledge proof for*

$$\mathcal{R}_{\pi_A} = \left\{ ((\text{ctxt}, T, A_1, A_2); (t, c, a_c)) : c \in \{1, 2\}, \quad T = tG, \quad A_c = a_cG, \quad \text{ctxt} = a_c + f(t) \right\}.$$

In words: the adaptor point T has discrete log t ; the ciphertext decrypts under the pad to a scalar a_c whose public image A_c is one of the two published thimbles; and the branch c (hence the outcome) is hidden.

Note that we refer to a generic function f acting on the secret value t ; this, and the actual concrete instantiation of π_A , are discussed in Section 4.

π_A represents Bob’s guarantee that when the adaptor is completed, he will be able to convert ctxt into a_c , Alice’s actually chosen discrete log, via the relation:

$$a_c = \text{ctxt} - f(t).$$

Once Bob has verified both the adaptor and the proof π_A , he is now safe that the bet is fair: he sends σ_B , which is his partial signature for the PayoutTx.

Once Alice receives σ_B , she is safe to broadcast the transaction, which:

- reveals σ , the total valid BIP340 signature on U_1 ;
- which Bob sees, and he calculates $t = \sigma - \sigma_B - \sigma'$;

- he then calculates $a_c = \text{ctxt} - f(t)$;
- he then compares $W_b + a_c G$ with K ; if it is the same, he has the secret key $w_b + a_c$ of O_K 's internal key and can spend by the key path;
- if he does not spend before the relative timelock t_1 , Alice spends via the CSV path, or uses a cooperative optimization — see Section 5.

3.3. The protocol as a sequence of messages

We assume that one peer has identified another candidate peer and there is a reliable transport via which to send messages in both directions. Alice is called the “Dealer” and Bob is called the “Player”. The absolute timelock T_R for the RefundTx must be much larger than the relative timelock T_1 for the PayoutTx's destination scriptPubKey so that the contract cannot be backed out of before its expiration.

1. Dealer sends funding information: u_A for funding FundingTx, P_A for MuSig2 destination key M (will be outpoint u_1), receiving address for RefundTx R_A , change address for PayoutTx C_A , coop-overlay address O_A , nonces for RefundTx and PayoutTx and CoopOverlay signature negotiation. Simultaneously, she sends A_1, A_2 and a proof of knowledge of the dlogs a_1, a_2 , keeping the latter secret.
2. Player validates the PoK of A_1, A_2 . Player sends funding information: u_B for funding FundingTx, P_B for MuSig2 destination key M , (will be outpoint u_1), receiving address for RefundTx R_B , change address for PayoutTx C_B , nonces for RefundTx and PayoutTx and CoopOverlay signature negotiation, and K for settlement transaction, along with a ZKP π_B proving that K is of the form $A_y + W_b$ for his choice y from A_1, A_2 and a further key he proves knowledge of (see above for details).
3. Dealer validates π_B . Dealer calculates MuSig2 destination address M and outpoint u_1 , and full FundingTx ($u_A, u_B \rightarrow u_1, C_B$). Dealer then calculates RefundTx ($u_1 \rightarrow R_A, R_B$) and calculates partial sig on RefundTx $\sigma_{A,R}$. Sends $\sigma_{A,R}$.
4. Player calculates MuSig2 destination address M and outpoint u_1 , and full FundingTx ($u_A, u_B \rightarrow u_1, C_B$). Player then calculates RefundTx ($u_1 \rightarrow R_A, R_B$). Having received $\sigma_{A,R}$ he now calculates his own partial sig on RefundTx $\sigma_{B,R}$. Sends $\sigma_{B,R}$.
5. Dealer validates RefundTx with the given full MuSig2 signature $\sigma_{A,R} + \sigma_{B,R}$. If all validates, she signs the FundingTx (which she already calculated) on u_A to get σ_A, F and passes this as a PSBT to Player.
6. Player validates RefundTx with the given full MuSig2 signature $\sigma_{A,R} + \sigma_{B,R}$. If all validates, he co-signs the FundingTx (which he already calculated) on u_B to get σ_B, F and combines this with the PSBT to broadcast the FundingTx.
7. Both sides wait for confirmation of funding transaction. Note that they already have the PayoutTx, which is ($u_1 \rightarrow O_K, C_A$)
8. Dealer calculates the adaptor signature and point $\sigma'_{A,P}, T$ for PayoutTx signing on u_1 (noting that the nonces were already exchanged for this MuSig session). She then calculates π_A and ciphertext ctxt as described above and sends $\sigma'_{A,P}, T, \pi_A, \text{ctxt}$. T was chosen so

that $\text{ctxt} = a_c + t^2 \bmod p$ and $tG = T$ and $a_c \in a_1, a_2$ and π_A proves this.

9. Player verifies $\sigma'_{A,P}, T, \pi_A$ against PayoutTx and the predetermined nonces. If it verifies, he calculates, and then sends $\sigma_{B,P}$.
10. Dealer constructs fully signed PayoutTx, by calculating $\sigma_{A,P}$. She sends t and her co-signature $\sigma_{A,O}$ on the CoopOverlayTx to the Player (but does *not* broadcast PayoutTx).
11. Player sees t , and uses it to calculate $a_c = \text{ctxt} - t^2 \bmod p$. And then compares $a_c G$ with A_y , if they match, he has won and will calculate the full MuSig2 cosignature on PayoutTx as $\sigma_{B,P} + \sigma'_{A,P} + t$ and use it to broadcast PayoutTx. And must send another payment from O_K to his own wallet independently before the timeout clause t_1 .
12. If they do not match, he calculates CoopOverlayTx, and his co-signature $\sigma_{B,O}$ on that transaction, and broadcasts it (knowing he has lost the bet anyway, he does this to maintain a payment-shaped transaction).
13. In case the Player disappears and does not complete the previous step promptly, the Dealer broadcasts, PayoutTx which pays u_1 into an output she will control after t_1 blocks⁴.

In cases where the steps say “X validates Y”, it is to be understood that if validation fails, that participant aborts, either by doing nothing, or, if they have already committed funds, by broadcasting RefundTx as is obvious according to context. Step 13 is a specific kind of abort; the bet completes, but some privacy w.r.t. outside observers is lost.

3.4. Extension to other payout probabilities

We omit this here, and claim it is a trivial extension, though not one without limitations: instead of an OR condition over two possibilities, make it an OR condition over N possibilities. This can give Bob a $p = 1/n$ chance of success. (Probably k/N for some threshold k is also possible, but more complex). Worthy of note is that the simple sigma protocols shown here have linear scaling in communication over N , so this would be very non-ideal for large N , but SNARKs used as alternative proving mechanisms could overcome this. The earlier mentioned GTT work [9] uses an OPRF mechanism which also removes this $O(N)$ scaling.

3.5. Payment-shaped transactions

There is no **requirement** for any privacy outcome here; the protocol functions purely as a wager. However it is noteworthy that the specific design above has an extremely minimal onchain footprint, and so it is **the cheapest as well as the most private**. This fits in with taproot’s design principles (and indeed, offchain payments, also). So it is plausible that ordinary bets take this structure, even if the users don’t care about privacy (see the Introduction).

But there is one specific privacy outcome that falls out of the root design:

Because of the symmetrical safety/security of the cryptographic construct, a purely peer to peer protocol is possible. Because of *that*, dual funding is not only possible, but natural. Because of *that symmetry of roles*, a partial breakage of com-

⁴The player can “grief” here also, by waiting for this event even if he won; but as noted elsewhere, this is not a money gaining attack, it only destroys a mild privacy effect for both participants

mon input ownership heuristics [19] is inevitable. Because of *that*, this protocol has a natural privacy effect, though not a very strong one on a per-transaction basis (much stronger over the whole transaction graph of bets).

Shooting for a stronger privacy effect than that is possible but it isn't necessary. The on-chain half of that stronger effect — making the bet transactions read as ordinary payments — is described here; the network-layer half (covert negotiation and discovery) is deferred to Section 6.

The other thing needed for the “first layer defence” of having these bets look like normal payments, is to disguise the fact that cooperation is occurring (because in fact, that is the sense in which they are not normal payments — they *are* payments, the only thing different is that they are co-funded, which breaks CIOH).

The PayoutTx spends a single input U_1 (it can optionally spend more but that's more complex) to a standard P2TR destination. It should also have a change output, which it naturally doesn't have.

The FundingTx has a different-shaped problem: its default structure has each party contribute one or more inputs, and the output U_1 is then accompanied by *two* change outputs. This is far from ideal, as 3-output transactions are far from common; we want a two-output transaction. The natural way to achieve this is for the dealer to provide a utxo which is equal to their suggested bet size (optionally, the player could do the same); this way they have no change output. But that is very restricting.

Fortunately, both problems are resolved by one idea: **spread the two change outputs across the two transactions**. The dealer (Alice) contributes her *entire* input to the pot and takes no change in the funding transaction; her change is instead released as the second output of the PayoutTx. The player (Bob) takes his change in the funding transaction as usual. Writing F_A, F_B for the two funding inputs, a, b for the (equal) stakes, $S = a + b$ for the at-risk pot, and $c_A = F_A - a$, $c_B = F_B - b$ for the two change amounts, the pot is $U_1 = F_A + b = S + c_A$.

The choice of *which* change goes in *which* transaction is deliberate. The Bob-wins branch must complete **without any cooperation from Alice** (she has no incentive to help a winning counterparty), so we want that branch to have the payment-like two-output shape *automatically*. Because Alice's change is an output of the *pre-signed* settlement, the Bob-wins settlement is two-output regardless of the outcome and regardless of cooperation (Figure 1); Bob then spends his claim output O_K by an ordinary key path. The Alice-wins branch (Figure 2) is the cooperative key-path spend of Section 5, which the parties are jointly motivated to use.

In this way, both Alice and Bob receive a change output from the execution of the protocol, unconditionally whether they won or lost the bet, and all the “happy-path” transactions have the two-output profile that is dominant in the distribution of onchain transactions, achieving the covertness property desired.

We note also the somewhat trivial points that nLockTime and nSequence should align with typical network behavior.

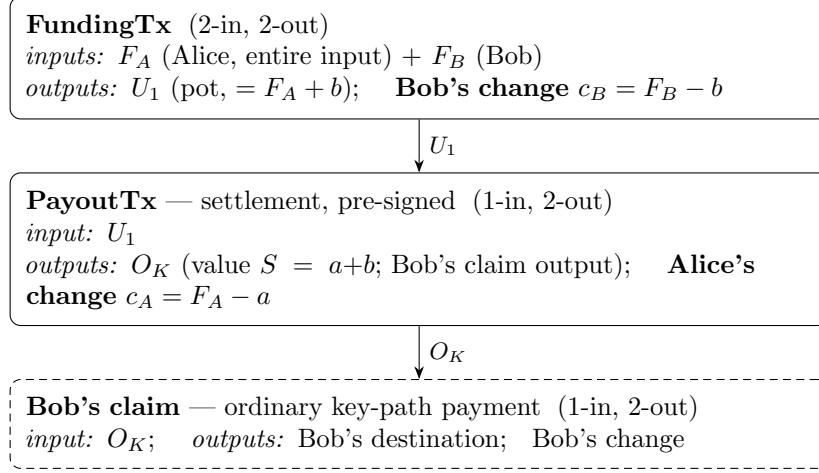


Figure 1: **Bob wins.** Alice's change rides in the pre-signed settlement, so that transaction is two-output automatically — no cooperation from Alice required. Every transaction has the dominant two-output shape.

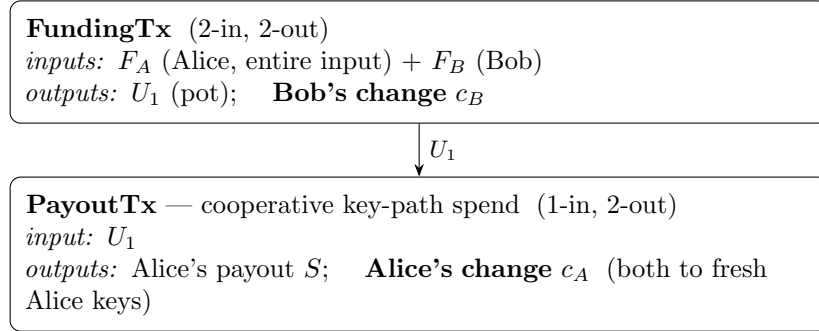


Figure 2: **Alice wins.** Rather than the settlement (which would force a script-path reclaim of O_K), the parties co-sign a single key-path spend of U_1 ; Alice's two outputs go to fresh keys and read as payment + change.

3.6. A pot-sweetener example

To illustrate the idea from Section 1.1, we show a case of Alice, the dealer, incentivizing Bob, the player. Alice starts with a utxo of size 0.2 and Bob with one of 0.8. Alice decides that the pot contributions should be size 0.05, but she also adds a sweetener of 0.01 for Bob to slightly favor him. Then we have:

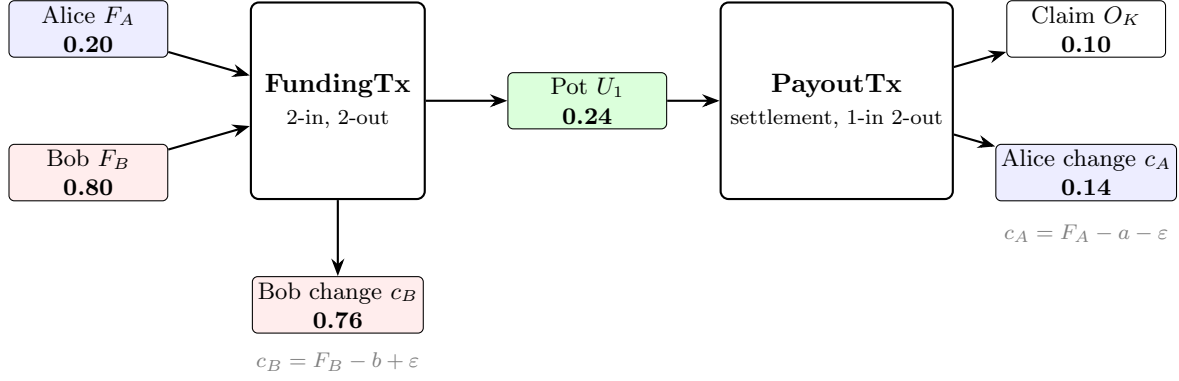


Figure 3: Pot-sweetener example (§3.6). Equal stakes $a = b = 0.05$ give an at-risk pot $S = a + b = 0.10$; Alice adds a sweetener $\varepsilon = 0.01$ favouring Bob. Alice funds $F_A = 0.20$, Bob funds $F_B = 0.80$. The sweetener is borne entirely by the two change outputs — Bob’s grows by ε , Alice’s shrinks by ε — so the intermediate pot becomes $U_1 = F_A + b - \varepsilon = 0.24$ while the at-risk claim $O_K = S = 0.10$ is unchanged. In expectation Alice forgoes ε and Bob gains ε .

4. Construction of the proof π_A

π_A proves the relation $\mathcal{R}_{\pi_A}$ of Definition 2 for a chosen “pad” function f . We’ll present a line of reasoning leading to a sensible choice for f .

4.1. The broken case: $f(t) = t$

The fact that this doesn’t work is instructive. If we write $\text{ctxt} = a_c + t \bmod p$, it seems plausible, because the dlog a_c is computationally hidden by a random field element t . But in fact in this case Bob can use the ciphertext to immediately derive the information on whether his guess was correct, without receiving the adaptor completion (i.e. without needing to wait until he receives the dlog t). The problem is that this relation is **linear**: he can compute $\text{ctxt}G - T = (a_c + t)G - tG = a_cG = A_c$, directly recovering *which* thimble Alice encrypted — hence her choice c — and comparing it against his guess A_y he learns the outcome before the adaptor completes, so he can abort if he is losing.

This is a consequence of the homomorphism of the group operation. The insight that follows is, f needs to be non-affine. Which leaves a substantial open question about whether it needs more structure than that. Hence the next option:

4.2. Squaring pad: $f(t) = t^2$

Take $f(t) = t^2$, so $\text{ctxt} = a_c + t^2$. Define the branch targets $Y_i := \text{ctxt}G - A_i$ for $i \in \{1, 2\}$. On the true branch,

$$Y_c = (\text{ctxt} - a_c)G = t^2G = t(tG) = tT,$$

so the entire relation $\mathcal{R}_{\pi_A}$ collapses, for this f , to

$$\exists c \in \{1, 2\}, t \in \mathbb{F}_p : \quad T = tG \quad \wedge \quad Y_c = tT.$$

This is a 1-of-2 CDS OR [13] of Chaum–Pedersen equal-discrete-log proofs [14], $\text{DLEQ}(G, T; T, Y_c)$: one witness t , proved to be simultaneously the discrete log of T base G and of Y_c base T .

4.2.1 Squaring proof construction

- Choose $a_i \xleftarrow{\$} \mathbb{Z}_p$ and the adaptor secret $t \xleftarrow{\$} \mathbb{Z}_p$ (the a_i were chosen before PayoutTx construction by Alice, but we emphasize the source of their choice here, as it is crucial to the security argument).
- Given the choice c from 1, 2 by Alice, construct $T = tG$ and the ciphertext $\text{ctxt} = a_c + t^2 \bmod p$.
- Construct $Y_i = \text{ctxt}G - A_i$, $i \in 1, 2$
- Choose commitment nonce $r_c \xleftarrow{\$} \mathbb{Z}_p$ (remember that c is Alice's choice from 1, 2) and note the corresponding commitment points $R_{c,T} = r_c T$, $R_{c,G} = r_c G$.
- Deduce commitment points for the fake branch of the CDS proof, i.e. for $i \neq c$. Choose : $e_i \xleftarrow{\$} \mathbb{Z}_p$ and $s_i \xleftarrow{\$} \mathbb{Z}_p$. Then set $R_{i,T} = s_i T - e_i Y_i$ and $R_{i,G} = s_i G - e_i T$.
- Calculate the Fiat-Shamir hash challenge: $e = H(\text{DOM}, A_1, A_2, T, \text{ctxt}, R_{..})$ where $R_{..}$ stands in for all 4 of the aforementioned commitment values. Here "DOM" is intended to stand in for some domain-tagging string.
- We now apply the CDS OR-proof idea: the challenges used for the two statements under OR must add (mod p) to the FS-challenge: $e_1 + e_2 = e \bmod p$. This gives us a specific value for e_c .
- Complete the sigma protocol for DLEQ on the "honest" branch: $s_c = r_c + e_c t \bmod p$.
- Publish the ciphertext and proof as $\text{ctxt}, T, R_{..}, s_1, s_2, e_1, e_2$.

(A bandwidth optimization would be to remove the $R_{..}$ values and only publish the challenges e , but this is an implementation detail).

4.2.2 Squaring proof verification

On receipt of the tuple $\text{ctxt}, T, R_{..}, s_1, s_2, e_1, e_2$, Bob does the following:

- He first reconstructs both Y_1 and Y_2 . These are $Y_i = \text{ctxt}G - A_i$. Notice how, unlike the broken linear case, this does *not* allow him to deduce the correct choice: $Y_c = tT$ but by the DDH assumption⁵, Bob cannot distinguish between the correct and wrong choice.
- He reconstructs the Fiat-Shamir challenge: $e = H(\text{DOM}, A_1, A_2, T, \text{ctxt}, R_{..})$
- He verifies on the T -branch: $s_i T \stackrel{?}{=} R_{i,T} + e_i Y_i$, $i \in 1, 2$
- He verifies on the G -branch: $s_i G \stackrel{?}{=} R_{i,G} + e_i T$, $i \in 1, 2$
- He verifies $e = e_1 + e_2 \bmod p$

⁵technically it's the square DDH assumption, because it's one exponent t repeated, not two

4.2.3 Squaring proof security: hiding

Security of the branch-hiding rests on a square-DDH assumption (distinguishing (G, tG, t^2G) from $(G, tG, \text{random}G)$). One caveat is intrinsic to squaring: the mask t^2 ranges over quadratic residues, so for a *guessable* candidate a the value $\text{ctxt} - a$ can be tested for quadratic residuosity, passing always for the true a and with probability $\approx \frac{1}{2}$ for a random false one. The hiding property is therefore sound only for *high-entropy* masked scalars — which the thimble scalars a_1, a_2 and t are, being sampled uniformly from $\mathbb{F}_p$. It must not be used to hide low-entropy plaintexts (amounts, counters, votes). See Appendix A for the detailed reduction proof.

Note that the proof would be totally unsound in a DDH-easy group (because DDH easy $\Rightarrow$ square-DDH easy); but secp256k1 is a DDH-hard group, because it is prime order and has no efficiently-computable pairing.

An additional point is that square-DDH is *not* reducible to DDH, or at least no such reduction is known.

4.2.4 Squaring proof security: soundness

The soundness of the proof is relatively simple; it's 2-special soundness in the ROM of the relevant hash function:

Lemma 1 (Knowledge soundness of π_A). *The squaring-pad proof π_A of Section 4.2 is a proof of knowledge for the relation*

$$\mathcal{R}_{\pi_A} = \{(\text{ctxt}, T, A_1, A_2); (t, c, a_c) : c \in \{1, 2\}, T = tG, A_c = a_cG, \text{ctxt} = a_c + t^2 \bmod p\}.$$

Sketch. Recall that π_A is a 1-of-2 CDS-OR of Chaum–Pedersen proofs of equal discrete logarithms:

$$\text{DLEQ}(G, T; T, Y_i), \quad Y_i = \text{ctxt} \cdot G - A_i, \quad i \in \{1, 2\}.$$

Each branch is a standard proof of knowledge of t satisfying $tG = T$ and $tT = Y_i$, and therefore has 2-special soundness: from two accepting transcripts with the same commitments but different challenges, an extractor recovers

$$t = \frac{s_i - s'_i}{e_i - e'_i} \bmod p.$$

The CDS-OR composition preserves this property. Given two accepting transcripts with distinct total challenges $e \neq e'$, where $e = e_1 + e_2$ and $e' = e'_1 + e'_2$, at least one branch has $e_i \neq e'_i$. Special soundness on that branch extracts t (note that this is true independent of which branch is extracted; the corresponding secret in both cases is t).

Once t is known, the ciphertext determines the encrypted thimble scalar:

$$a_c = \text{ctxt} - t^2 \bmod p.$$

Checking which published thimble satisfies $a_cG = A_c$ identifies the branch c and completes the witness. $\square$

4.3. ZK-friendly hash pad: $f = \text{Poseidon}$

This is speculative, and would need the input of experts, though at a high level, it is very clearly possible.

Taking f to be a hash makes $\text{ctxt} = a_c + f(t)$ a non-affine relation. We prove it in zero knowledge with a transparent Bulletproofs [15] R1CS instantiated over secp256k1 (using the Curve Trees generic-curve backend [16]), where f is a Poseidon [17] permutation over $\mathbb{F}_p$ — field-native, with the x^5 S-box (valid since $\gcd(5, p-1) = 1$) — together with a sigma conjunct binding a_c to a published thimble and t to T . This gives a general (any hash-like f) route at higher cost: a few hundred multiplication gates, i.e. order 10^2 ms to prove and a ~ 1 KB proof, still with no trusted setup. A reviewed, field-native alternative to a hand-parameterised Poseidon is the Purify PRF of MuSig-DN [18], an algebraic PRF whose output already lands in the circuit’s scalar field.

In summary:

The squaring construction is complete and cheap but couples f to t^2 ; the hash construction admits an arbitrary f at a cost. Which is preferable — and whether reference parameters or a reviewed PRF should replace the hand-rolled Poseidon — is left to future work.

5. Cooperative Alice-payout completion

The enforced payout path of the simplest description of Section 3 leaves exactly one non-payment-like footprint: when Bob loses, Alice reclaims O_K through the CSV script leaf, revealing a tap-script rather than spending by a key path. We have avoided this problem in our detailed protocol description – in this section we’ll describe the reasoning behind the CooperativeOverlayTx that enables this.

Once the outcome is determined, rather than broadcasting the PayoutTx, Alice sends t to Bob off-chain. Bob checks $tG = T$, computes $a_c = \text{ctxt} - f(t)$, and — finding $a_cG \neq A_y$, i.e. that he lost ($y \neq c$) — co-signs with Alice a plain MuSig2 key-path spend of the pot U_1 directly to a fresh destination of Alice’s. No PayoutTx ever reaches the chain: the new payout (“Cooperative Overlay”) is an ordinary taproot key-path payment, indistinguishable from a normal transfer.

Bob has no incentive to refuse: he has lost regardless, so withholding his signature only delays Alice, and the enforced settlement-then-CSV path remains available as a backstop if he stalls. Thus in the common (honest) case *both* outcomes — Bob-wins (already a key-path spend of O_K) and Alice-wins (this cooperative key-path spend of U_1) — are plain payments.

6. Network communication layer

Two network-layer needs remain: an off-chain channel over which the two parties negotiate the protocol, and a way to *discover* counterparties in the first place. Both can be met by mechanisms that are themselves covert — so that not even the existence of a negotiation is visible to third parties — pairing naturally with the payment-shaped on-chain footprint of Section 3.5.

6.1. BIP324 decoy packets

If BIP324 [22] P2P encryption is used — as it now is for the majority of global Bitcoin P2P traffic, having been enabled by default since Bitcoin Core 27.0 [23] — then it is possible for individual nodes to send decoy packets that are not distinguishable from the true P2P traffic. This is one way for first-class Bitcoin users (that is, users who actually fully verify payments with their own node) to negotiate the Babilonia protocol to bet with each other trustlessly, and in theory, without anyone else knowing they’re doing it.

It’s exactly in this use-case scenario where it would be especially advantageous to have the onchain trace of the betting protocol look purely like payments.

To support this at the network layer, a very simple patch to Bitcoin Core may be enough.⁶ Having said this, it’s obvious that this is far from the only way to exchange messages privately! It’s just one that’s quite serendipitous: it aligns with privacy and security conscious bitcoin usage, and even provides a slight incentive to run more Bitcoin nodes, which is healthy for the network.

6.2. Discovery and Sybil attacks

Both in the case of communicating full nodes or Babilonia users communicating otherwise, there is the natural problem of how to discover counterparties. We leave a concrete rendezvous mechanism to future work.

As an extreme, consider that sending BIP324 decoy requests, that are ignored by vanilla Core nodes, but considered and potentially accepted by Babilonia-patched nodes, allows anonymous parties to engage in this protocol. The fact that this is even possible is of course a direct result of the fully trustless fair bet protocol created by the cryptography.

Naive implementation is, like any other anonymous protocol, open to Sybil attacks. This can be addressed with cost imposition (amongst other more brittle solutions) - a huge topic that we won’t address here.

7. Example implementation

See <https://github.com/AdamISZ/babilonia>. The code is largely written by Claude Opus 4.8 as of now, and therefore carries a “do not use with real money” warning, though it would need review in any case. However, it has been tested on signet as being functional. To use, patch 2 or more Bitcoin Core nodes with the provided patch (or simply test locally on regtest).

As of the time of writing, this implementation has no “pot-sweetener” function, but it is not hard to add.

8. Other similar ideas

If we take the core idea here as “not-balance-preserving but cryptographically fair collaborative transactions” and, as noted, this can have some significant benefits in terms of obscuring

⁶A proof-of-concept patch adding `senddecoy/getdecoys` RPCs is included with the reference implementation [25].

transaction history, there are a few different other directions one could look at it, than this very simple “probabilistic coinjoin with covert properties”. For example:

- The same but applied to 2 party privacy perserving coinswaps [26]: their main disadvantage is their equal-amount fingerprint, which had previously been addressed with splitting. This construction, or that of GTT [9], could address this.
- A not-covert (i.e. no steganographic) large scale coinjoin a la Wasabi or similar, but with an inbuilt lottery. This could incentivize (possibly?) usage of larger user groups. Achieving that outcome trustlessly seems non-trivial
- Just a lottery generally, in one of these or another form
- Probabilistic swaps may have several other applications as was briefly reviewed in [9]

... and perhaps many others I haven’t thought of.

References

- [1] D. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM*, 24(2):84–90, 1981.
- [2] G. Maxwell. CoinJoin: Bitcoin privacy for the real world. BitcoinTalk, 2013. <https://bitcointalk.org/index.php?topic=279249.0>
- [3] Á. Ficsor et al. ZeroLink: The Bitcoin fungibility framework, 2017. (Basis of the Wasabi and Samurai/Whirlpool equal-output CoinJoins.)
- [4] Á. Ficsor, L. Hangyi, K. Kravchenko, G. Kajdǎ, Y. Sun, T. Babó. WabiSabi: Centrally coordinated CoinJoins with variable amounts. IACR ePrint 2021/206. <https://eprint.iacr.org/2021/206>
- [5] J. Poon and T. Dryja. The Bitcoin Lightning Network: Scalable off-chain instant payments, 2016. <https://lightning.network/lightning-network-paper.pdf>
- [6] N. Dorier. BIP 78: A simple Payjoin proposal, 2019. <https://github.com/bitcoin/bips/blob/master/bip-0078.mediawiki>
- [7] Satoshi Dice. Bitcoin Wiki. https://en.bitcoin.it/wiki/Satoshi_Dice
- [8] O. Kurbatov. Emulating OP_RAND in Bitcoin. arXiv:2501.16451, 2025. <https://arxiv.org/abs/2501.16451>
- [9] P. Gerhart, J. Taylor, S. A. K. Thyagarajan. Probabilistic Atomic Swaps for Bitcoin and Friends. arXiv:2605.04975, 2026. <https://arxiv.org/abs/2605.04975>
- [10] J. Nick, T. Ruffing, Y. Seurin. MuSig2: Simple two-round Schnorr multi-signatures. *CRYPTO 2021*; IACR ePrint 2020/1261. <https://eprint.iacr.org/2020/1261>
- [11] J. Nick, T. Ruffing, Y. Seurin. BIP 327: MuSig2 for BIP340-compatible multi-signatures, 2022. <https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki>
- [12] L. Fournier. One-time verifiably encrypted signatures, a.k.a. adaptor signatures, 2019.

- <https://github.com/LLFourn/one-time-VES/blob/master/main.pdf>
- [13] R. Cramer, I. Damgård, B. Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. *CRYPTO 1994*, LNCS 839, 174–187.
 - [14] D. Chaum, T. P. Pedersen. Wallet databases with observers. *CRYPTO 1992*, LNCS 740, 89–105.
 - [15] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. *IEEE S&P 2018*. <https://eprint.iacr.org/2017/1066>
 - [16] M. Campanelli, M. Hall-Andersen, S. E. Kamp. Curve Trees: Practical and transparent zero-knowledge accumulators. *USENIX Security 2023*. <https://eprint.iacr.org/2022/756>
 - [17] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. *USENIX Security 2021*. <https://eprint.iacr.org/2019/458>
 - [18] J. Nick, T. Ruffing, Y. Seurin, P. Wuille. MuSig-DN: Schnorr multi-signatures with verifiably deterministic nonces. *ACM CCS 2020*; IACR ePrint 2020/1057. <https://eprint.iacr.org/2020/1057>
 - [19] S. Meiklejohn, M. Pomarole, G. Jordan, K. Levchenko, D. McCoy, G. M. Voelker, S. Savage. A fistful of Bitcoins: characterizing payments among men with no names. *ACM IMC 2013*. <https://cseweb.ucsd.edu/~smeiklejohn/files/imc13.pdf>
 - [20] P. Wuille, J. Nick, A. Towns. BIP 341: Taproot: SegWit version 1 spending rules, 2020. <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>
 - [21] A. Davidson, A. Faz-Hernández, N. Sullivan, C. A. Wood. RFC 9497: Oblivious Pseudorandom Functions (OPRFs) using prime-order groups, 2023. <https://www.rfc-editor.org/rfc/rfc9497>
 - [22] P. Wuille, D. Mehta, T. Ruffing, J. Schnelli. BIP 324: Version 2 P2P encrypted transport protocol. <https://github.com/bitcoin/bips/blob/master/bip-0324.mediawiki>
 - [23] Bitcoin Optech. Version 2 P2P transport. <https://bitcoinops.org/en/topics/v2-p2p-transport/>
 - [24] Bitcoin Optech. Fee sniping. <https://bitcoinops.org/en/topics/fee-sniping/>
 - [25] Babilonia reference implementation. <https://github.com/AdamISZ/babilonia>
 - [26] Citadel Tech. Coinswap: a trustless, peer-to-peer implementation of the Maxwell–Belcher CoinSwap protocol. <https://github.com/citadel-tech/coinswap>
 - [27] J. L. Kelly Jr. A new interpretation of information rate. *Bell System Technical Journal*, 35(4):917–926, 1956.
 - [28] waxwing. Payjoin blog post Jan 2019 <https://web.archive.org/web/20201108034458/https://joinmarket.me/blog/blog/payjoin/>

[29] Joinmarket, original author Chris Belcher. See <https://en.bitcoin.it/wiki/JoinMarket>

9. Appendix A: Reduction of the squaring proof to square-DDH

We assume the existence of an adversary $\mathcal{A}^{WFP}$ who with non-negligible probability can, given the output the protocol of section 4.2.1-4.2.3: $\text{ctxt}, T, \sigma'_{A,P}$ with the pre-determined thimbles A_1, A_2 , whether the choice was $c = 1$ or $c = 2$.

We show that this adversary solves the square-DDH problem with non-negligible probability.

Square Decisional Diffie–Hellman (sqDDH). Let G be a group of prime order p with generator G , written additively. The *square-DDH* advantage of an adversary $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{sqDDH}}(p) = \left| \Pr[\mathcal{A}(G, tG, t^2G) = 1 \mid t \xleftarrow{\$} \mathbb{Z}_p] - \Pr[\mathcal{A}(G, tG, uG) = 1 \mid t, u \xleftarrow{\$} \mathbb{Z}_p] \right|.$$

The *square-DDH assumption* holds in G if for every probabilistic polynomial-time adversary $\mathcal{A}$, this advantage is negligible in $\log p$.

Game Square-DDH:

Game sqDDH $_{\mathcal{G}, \mathcal{A}}(\lambda)$. Let $\mathcal{G} = (G, p, G)$ be a description of a cyclic group of prime order q generated by G , sampled by $\text{GroupGen}(1^\lambda)$, where λ is the security parameter. The game proceeds between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ as follows:

1. **Setup.** The challenger runs $\mathcal{G} \leftarrow \text{GroupGen}(1^\lambda)$, obtaining (G, p, G) , and samples $t \xleftarrow{\$} \mathbb{Z}_p$.
2. **Challenge.** The challenger samples a bit $b \xleftarrow{\$} \{0, 1\}$.
 - If $b = 0$ (*real*), set $Y \leftarrow t^2G$.
 - If $b = 1$ (*random*), sample $u \xleftarrow{\$} \mathbb{Z}_p$ and set $Y \leftarrow uG$.

The challenger sends the tuple (G, p, G, T, Y) , where $T \leftarrow tG$, to the adversary $\mathcal{A}$.

3. **Guess.** $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$.

Winning condition. $\mathcal{A}$ wins the game if $b' = b$. We define the advantage of $\mathcal{A}$ in the square-DDH game as

$$\text{Adv}_{\mathcal{G}, \mathcal{A}}^{\text{sqDDH}}(\lambda) = \left| \Pr[\text{sqDDH}_{\mathcal{G}, \mathcal{A}}(\lambda) = 1] - \frac{1}{2} \right|.$$

The *square-DDH assumption* holds with respect to GroupGen if for every probabilistic polynomial-time adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{G}, \mathcal{A}}^{\text{sqDDH}}(\lambda)$ is negligible in λ .

We define the entity C^{sqddh} as the challenger for Game sqDDH. We define the entity R as the adversary for Game sqDDH and as challenger for Game WFP. And we define $\mathcal{A}^{WFP}$, as noted, as the adversary assumed to be able to win Game WFP. As shown in Table 1 R can use $\mathcal{A}^{WFP}$ to win Game sqDDH by programming the RO for $\mathcal{A}^{WFP}$. Note that R must choose a_1, a_2 and ctxt randomly from $\mathbb{Z}_p$ so that the inputs to $\mathcal{A}^{WFP}$ are indistinguishable from a normal run of Game WFP. It's for this reason that the proof does *not* hold if any of the quantities a_1, a_2, t are not chosen purely at random from $\mathbb{Z}_p$ (for example, if they are chosen from an enumerable set).

9.1. Remarks

- The only values in the transcript that can leak c are the implicit Y_1 and Y_2 where $Y_i = \text{ctxt}G - A_i$. In the real case, exactly one of them is a sqDDH tuple (G, T, Y_i) , while the other is random. Distinguishing which is the sqDDH tuple is the sqDDH problem. All other transcript components (PoKs, adaptor signature, CDS OR proof) are either independent of c or perfectly simulated under the traditional (RO + HVZK) model.
- ROM requirement. The reduction programs two random oracles $\mathbb{H}_{zk}, \mathbb{H}_{sig}$ (for the CDS OR proof and the Schnorr PoKs) and (for the adaptor signature challenge) respectively. This is standard for Fiat–Shamir-based proofs.
- The false branch Y_j is always random. In both cases, $Y_j = \text{ctxt}G - a_jG$ where a_j is uniform and independent (guaranteed by the random choice of ctxt), so Y_j is a uniformly random element of the group, independent of b . This ensures that in the real case, only Y_c carries the sqDDH structure, and in the random case, neither does.

9.2. Tightness reduction

Theorem 1 (Tightness of the squaring-proof reduction). *Let $\mathcal{A}^{\text{WFP}}$ be an adversary that, given a honestly generated transcript $(A_1, A_2, \text{PoKs}, \text{ctxt}, T, \sigma'_{A,P}, R, s_1, s_2, e_1, e_2)$ of the squaring proof from [subsection 4.2](#), distinguishes the true branch $c \in \{1, 2\}$ with advantage*

$$\varepsilon = \Pr[c' = c] - \frac{1}{2},$$

where $a_1, a_2, t \xleftarrow{\$} \mathbb{Z}_p$ are sampled uniformly and independently. Then there exists a square-DDH adversary R , running in time comparable to that of $\mathcal{A}^{\text{WFP}}$ plus the cost of simulating one transcript (a constant number of group operations and random-oracle queries), such that

$$\text{Adv}_R^{\text{sqDDH}}(\lambda) \geq \frac{1}{2} \varepsilon.$$

Sketch. R embeds the sqDDH challenge point Y as $Y_c = \text{ctxt} \cdot G - A_c$ for a uniformly guessed $c \xleftarrow{\$} \{1, 2\}$, samples $\text{ctxt}, a_j \xleftarrow{\$} \mathbb{Z}_p$ (with $j = 3 - c$), sets $A_c = \text{ctxt} \cdot G - Y$ and $A_j = a_jG$, and simulates the Schnorr PoKs, the adaptor signature, and the CDS OR proof entirely by random-oracle programming; see [Table 1](#).

Real case ($b = 0, Y = t^2G = tT$): then $a_c = \text{ctxt} - t^2$ is uniform because ctxt is, and $Y_c = tT$ satisfies the DLEQ relation, so the simulated transcript is distributed identically to a real one with branch c . Hence $\Pr[b' = 0 \mid b = 0] = \Pr[c' = c \mid b = 0] \geq \frac{1}{2} + \varepsilon$.

Random case ($b = 1, Y = uG$): both Y_1, Y_2 are uniform and independent of T , the CDS simulation is identical for both branches, and c is information-theoretically hidden, so $\Pr[b' = 1 \mid b = 1] = \Pr[c' \neq c \mid b = 1] = \frac{1}{2}$.

Combining,

$$\Pr[R \text{ wins}] = \frac{1}{2}(\frac{1}{2} + \varepsilon) + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2} + \frac{\varepsilon}{2},$$

giving $\text{Adv}_R^{\text{sqDDH}}(\lambda) \geq \varepsilon/2$. □

The factor-2 loss comes solely from R 's blind guess of c .

Step	C^{sqddh} (sqDDH challenger)	R (sqDDH adversary & WFP challenger)	$\mathcal{A}^{\text{WFP}}$ (WFP adversary)
1	Samples $t \xleftarrow{\$} \mathbb{Z}_p$, $b \xleftarrow{\$} \{0,1\}$. If $b=0$: $Y \leftarrow t^2G$. If $b=1$: $u \xleftarrow{\$} \mathbb{Z}_p$, $Y \leftarrow uG$. Sets $T \leftarrow tG$. Sends (G, T, Y) .		
2		Receives (G, T, Y) . Guesses $c \xleftarrow{\$} \{1,2\}$; let $j = 3-c$ (the complement).	
3		Samples $\text{ctxt} \xleftarrow{\$} \mathbb{Z}_p$ and $a_j \xleftarrow{\$} \mathbb{Z}_p$. Sets $A_j \leftarrow a_jG$ and $A_c \leftarrow \text{ctxt} \cdot G - Y$, so that $Y_c = \text{ctxt} \cdot G - A_c = Y$ by construction.	
4		Simulates Schnorr PoKs for A_1, A_2 : honest for A_j (knows a_j); for A_c , programs the RO and simulates (does not know a_c).	
5		Simulates the adaptor signature $\sigma'_{A,P}$: samples $s' \xleftarrow{\$} \mathbb{Z}_p$ and $e_{\text{sig}} \xleftarrow{\$} \mathbb{Z}_p$; sets $R_A \leftarrow s'G + T - e_{\text{sig}}\mu_A P_A$; programs $H_{\text{sig}}(R_{\text{agg}}, P_{\text{agg}}, m_{\text{payout}}) \leftarrow e_{\text{sig}}$.	
6		Simulates the CDS OR proof (both branches). For each $i \in \{1,2\}$: samples $e_i, s_i \xleftarrow{\$} \mathbb{Z}_p$; computes $Y_i = \text{ctxt} \cdot G - A_i$; sets $R_{i,T} \leftarrow s_iT - e_iY_i$ and $R_{i,G} \leftarrow s_iG - e_iT$. Sets $e \leftarrow e_1 + e_2 \bmod p$. Programs $H_{\text{zk}}(\text{DOM}, A_1, A_2, T, \text{ctxt}, R_{\cdot}) \leftarrow e$.	
7		Sends the full transcript $(A_1, A_2, \text{PoKs}, \text{ctxt}, T, \sigma'_{A,P}, R_{\cdot}, s_1, s_2, e_1, e_2)$ to $\mathcal{A}^{\text{WFP}}$.	Receives the transcript; reconstructs $Y_1, Y_2 = \text{ctxt} \cdot G - A_i$; verifies the adaptor-signature equation $s'G \stackrel{?}{=} R_A - T + e_{\text{sig}}\mu_A P_A$ and the CDS OR equations $s_iG \stackrel{?}{=} R_{i,G} + e_iT$, $s_iT \stackrel{?}{=} R_{i,T} + e_iY_i$, $e_1 + e_2 \stackrel{?}{=} e$.
8			Outputs guess $c' \in \{1,2\}$.
9		If $c' = c$: outputs $b' \leftarrow 0$ ("real / sqDDH tuple"). If $c' \neq c$: outputs $b' \leftarrow 1$ ("random").	
10	R wins iff $b' = b$.		

Table 1: Reduction from WFP branch-distinguishing to sqDDH. R embeds the sqDDH challenge point Y as Y_c (the true-branch target) and simulates the remaining transcript entirely by RO programming. R never learns t or a_c .

10. Appendix B: Mathematics of betting on coinflips

10.1. Fixed-fraction betting (Kelly)

If you bet fraction f of your *current* balance each round, your balance follows a multiplicative random walk with median retention $\sqrt{1-f^2}$ per round. Over n rounds the median balance (assuming f^2 is small) is

$$B_0(1-f^2)^{n/2} \approx B_0 e^{-nf^2/2}.$$

The half-life is $\approx 2 \ln 2 / f^2$. For $f = 0.3$ this is ~ 15 rounds — catastrophic⁷. For $f = 0.01$ it's $\sim 13,860$ rounds — negligible. The drag is quadratic in f , so small fractions are strongly favored. This is the quantitative exposition of what's called the Kelly criterion [27]. If you don't like the mathematics, here's the words: if you have an edge, you should size your bets “in-line” with that edge. So don't bet 80% of your bankroll if your edge is 5%; surprisingly, you won't just reduce your winnings like that, you'll **actually turn a win into a loss** (over enough repeated games). Of course the “correct” bet fraction for a zero edge is 0%. How much you're damaged by betting more is quantified as per above. Look up the term “asymmetric leverage” for more counterintuitive facts about gambling, I mean, uh, “investment”.

We see here already, though, that *if* the fraction you bet is super-small, this “drag” effect or “asymmetric leverage” is barely noticeable at all. It's about big or medium sized bets, really.

10.2. Fixed-amount betting (additive walk)

Betting a fixed amount a avoids asymmetric leverage — the median equals the mean, $\mathbb{E}[B_n] = B_0$ in expectation — on average you lose nothing (we're ignoring tx fees here). To put it another way, after 100 rounds you're just as likely to be in the red as in the black, but only if we ignore another effect - *risk of ruin*. This is because 0 is not like any other balance: when you reach it, you can't bet any more.

The balance drifts as $\pm a\sqrt{n}$ (typical displacement), so after n rounds the balance is scattered in a window of width $\sim a\sqrt{n}$ around B_0 (the starting balance). Ruin probability after n rounds starting from B_0 is

$$\approx 1 - \Phi\left(\frac{B_0}{a\sqrt{n}}\right).$$

...where Φ is the CDF of the normal distribution. For small a/B_0 this is negligible over reasonable horizons; for larger fractions it becomes the binding constraint, *if* B_0 is really “your balance” (as in, all of your bitcoin).

But there's another much stronger reason to say “risk of ruin is not really important”; it's not only the using of small amounts a ; it's the fact that user balances are generally spread over many utxos, and they would be wise to only use such a system with smaller ones that represent a small chunk of their balance. Other technical factors apply: if you use a utxo with amount $\simeq a$ it's problematic to bet the whole of a anyway, since that means no change output or a dusty one.

⁷i.e. you lose half your money in 15 bets on average

10.3. Some vague thoughts on the effective cost of this as a coinjoin strategy

There's no clean measure of efficacy; the cost of this strategy either depends on f^2 or is notionally zero for fixed bet size. Volatility is not the same as cost, but as per some of the above, the cost of volatility itself is not zero. What's worse is, it's extremely unclear what exact privacy outcome you achieve with this strategy; steganographic outcomes are inherently unclear in anonymity set. To the extent that we are sticking to a 2-party protocol, it's not realistic to claim similar outcomes to standard coinjoin without many rounds⁸. What we *could* do is be optimistic and say that the upper bound anonymity set is N after N bets — note here how much vastly better coinswap is than payjoin here — and in this case, the cost imposed by the volatility is again linear in N .

But in any case, the effect of doing this is inherently different from that of standard coinjoin, as was explained in the introduction. We refrain from offering more equations as they probably will mislead the reader into thinking something more specific than they should.

The realistic strategy is probably small fixed amounts rather than small fractions, but with notable randomness, and accepting a bounded random-walk scatter as the cost of privacy, but the effect is difficult to measure.

⁸although the reader should not delude themselves in thinking that a toy model of k -anonymity achieved for a k -party coinjoin is really right either; the peripheral activity always confuses the issue